

Deliverable 1: The Artifact

What was built, and what it looks like running

Mills-Operation · AI Reliability Copilot for Hot Mill Operations

The Artifact

This deliverable walks through what was actually built, end to end, starting from where the data came from and finishing at the live console. Full detail on any one piece is in the other deliverables (architecture, data strategy, AI partnership log); this is the connective narrative across all of them.

There was no real data, so a realistic proxy was built first

Real Nucor sensor data was not available, so it had to be synthesized from scratch. Before writing a line of model code, the question was: what is the minimum realistic signal that actually proves a model can see a bearing failure coming before the alarm does.

The answer landed on 5 signals per roll stand, sampled every minute: vibration, bearing temperature, motor current, line speed, and coolant pressure. These were not picked because they were easy to fake. Between them they cover three different physical failure signatures (mechanical, thermal, electrical), one signal that reflects an operator's own response to a developing fault (line speed gets throttled back), and one that moves in the opposite direction from all the others (coolant pressure drops as the seal degrades). A model fed only correlated signals learns nothing new from most of them; this set was chosen specifically to avoid that.

The first version of the failure signature was lazy: flat normal noise, then a sudden step change right before the failure timestamp. It looked fake the moment it was plotted, no real machine dies like a light switch. It was rebuilt as a non linear ramp instead: starting 6 to 48 hours before the failure, each signal drifts toward its failing value on a curve that is slow at first and accelerates near the end, matching how bearing wear actually behaves, with the noise widening as the fault develops rather than just the mean shifting. `data/generate_synthetic_data.py` is fully seeded, so it is reproducible without checking a 25MB CSV into git history. It produces 6 stands times 45 days at 1 minute resolution, about 389 thousand rows, with roughly 22% of stand days ending in an injected failure, paired with a ground truth `failure_events.csv`.

Turning raw readings into something a model can actually use

A raw vibration reading of 9.5 mm/s means nothing on its own, it only means something relative to what that specific stand's normal running value looks like, and whether it is actively trending. `app/features.py` adds, for every signal, a 60 minute rolling mean and standard deviation (is this stand running hotter or noisier than its own recent normal) and a 15 minute rate of change (is it actively moving, not just sitting elevated). Deliberately simple on purpose, no frequency domain features, to get an honest end to end baseline working before reaching for anything fancier.

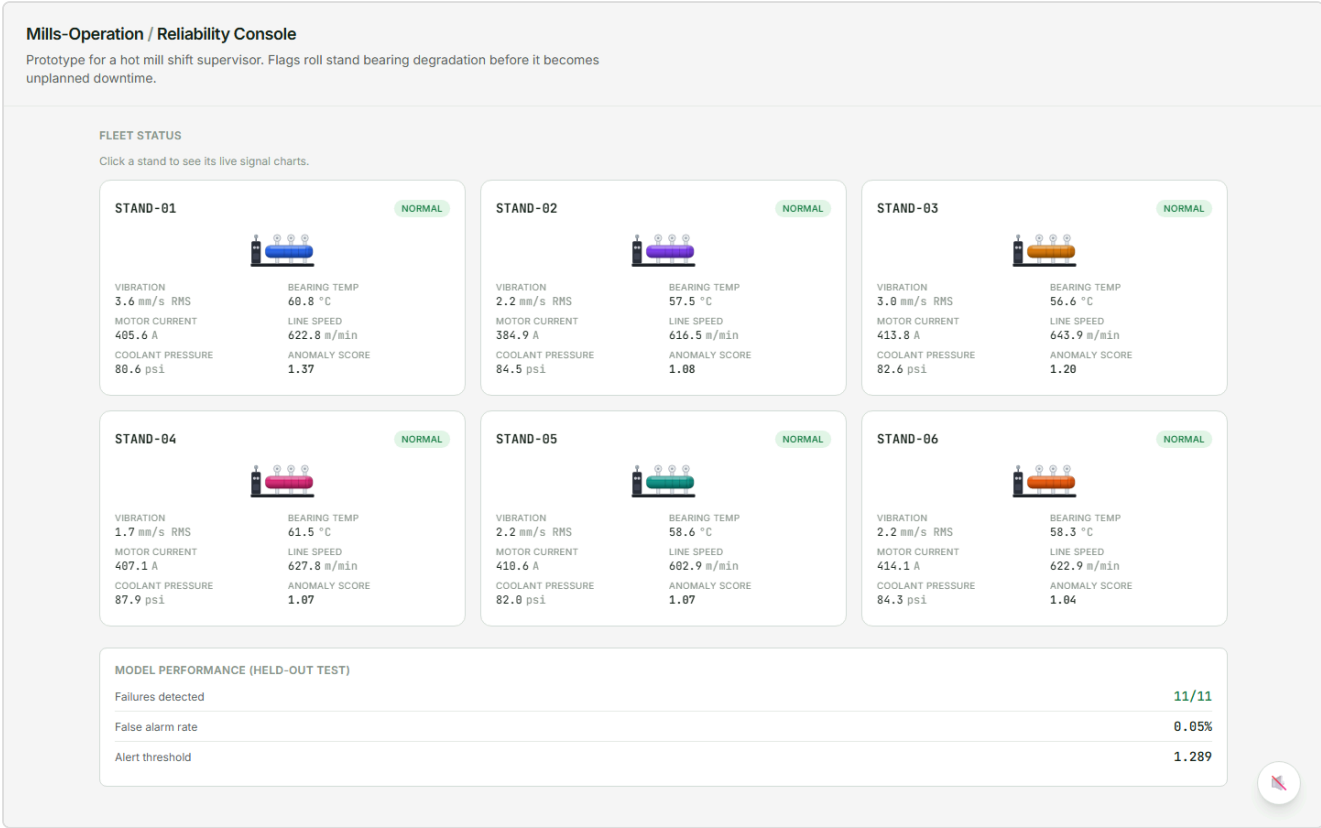
Training and evaluating the detector

The detector is a LocalOutlierFactor model, trained only on windows labeled clean normal (not approaching a failure, and not still recovering from one, both directions matter, see the data strategy deliverable for why). It was not picked by reasoning alone. `notebooks/model_comparison.ipynb` actually benchmarks it against IsolationForest, OneClassSVM, EllipticEnvelope, and a naive baseline, all trained on identical features and graded on identical lead time and false alarm metrics. LocalOutlierFactor won outright.

On the held out test window, the shipped model catches 11 out of 11 injected failures, with a 0.05% false alarm rate on normal minutes and a median lead time of roughly 687 minutes before the actual failure timestamp. An independent

backstop sits alongside the model itself: if a raw signal stays more than 3.5 standard deviations from a stable, whole training period baseline for 20 straight minutes, an alert fires regardless of what the model's own score says, since LocalOutlierFactor's local density approach has a real blind spot on failures that have already been going on for hours (its own recent broken minutes start looking normal to each other). Finding and fixing that blind spot is documented in full, including the labeling bug it led to, in the AI partnership log.

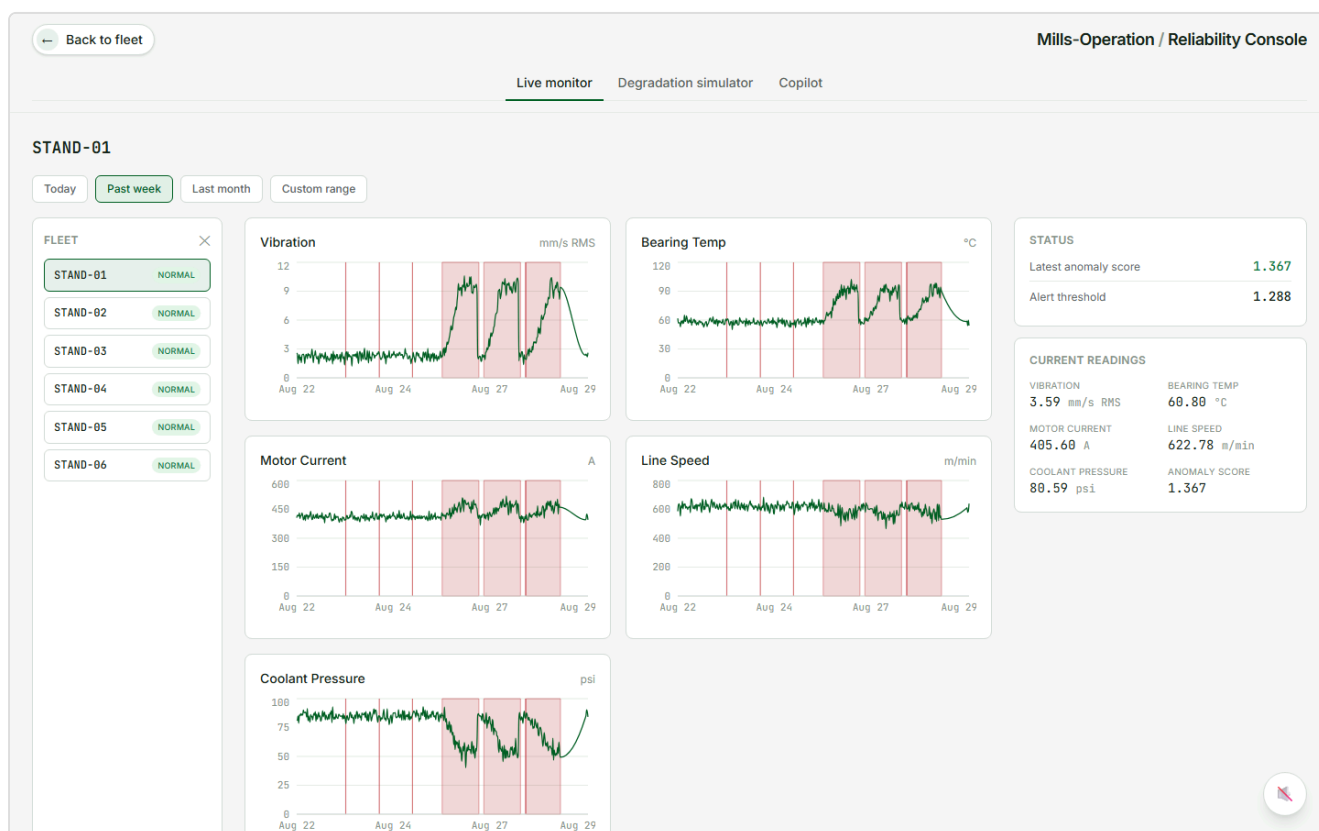
Those exact numbers, 11 out of 11, 0.05%, are not a claim, they are what the fleet screen below reads directly off the held out test run every time it loads.



The fleet landing page. All 6 stands with live sensor values on the left, and the model's actual held out test performance on the right, read straight from the evaluation run, not typed in by hand.

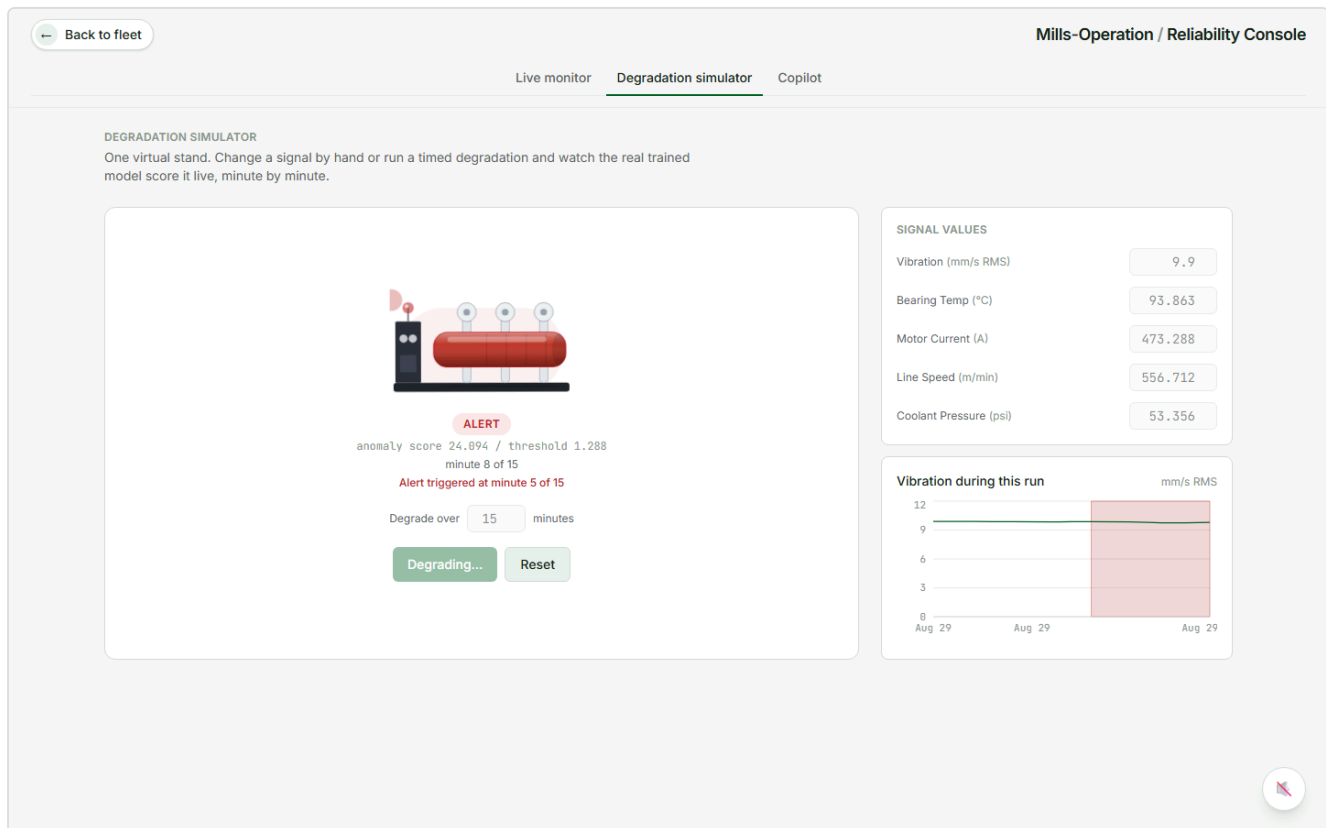
What actually runs live, not just a one time script

Training and evaluation happen once, but the console itself is live. `app/live_feed.py` ticks once every real minute, generates a new row per stand using the same baseline and failure ramp logic as the training generator, and scores it with the same trained model. `app/historical.py` keeps a rolling 45 day window of scored history that always ends yesterday and silently backfills itself if the app has been closed for a few days, so a date range filter on the dashboard (today, past week, last month, or custom) never shows a gap.



A single stand's signal history over the past week, filtered live from the dashboard. The shaded bands are real detected alert windows, not annotations added after the fact.

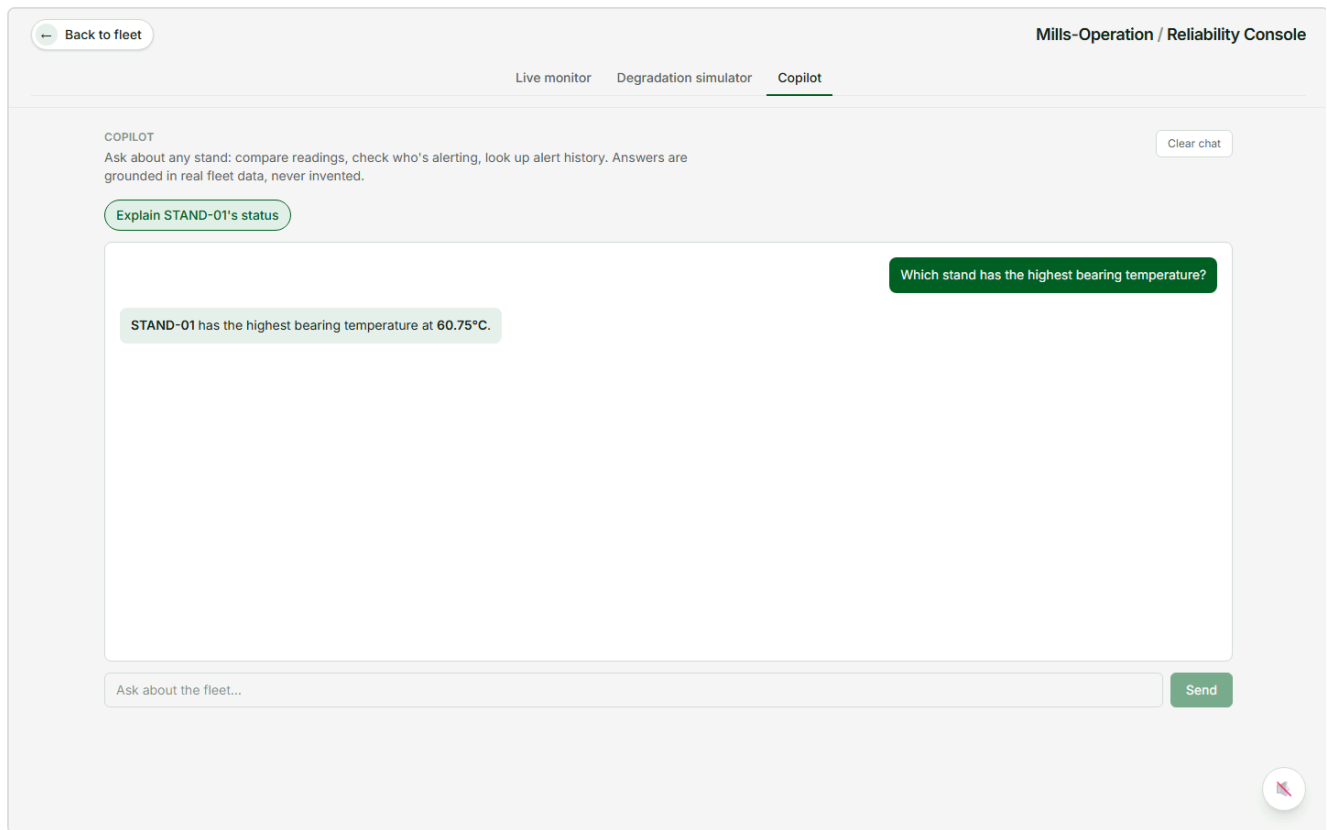
A separate degradation simulator lets a supervisor push a virtual stand toward failure by hand instead of waiting on the live feed's own randomness, and watch the real trained model react minute by minute as it happens.



Vibration pushed toward failure by hand on the virtual stand. The alert badge, the anomaly score, and the shaded band on the chart are all live output from the same detector used everywhere else in this console, not a scripted animation.

The AI layer is not a chatbot bolted on afterward

Two AI surfaces exist. A single stand explanation turns a flagged anomaly's ranked signal deviations into a plain language explanation and one concrete recommendation. A fleet wide copilot, on its own tab with real conversation memory, answers free text questions across all 6 stands (compare two stands, who is alerting, what is causing it, who is trending toward trouble next) by calling 6 narrow, read only Python functions and writing an answer only from what those functions actually returned. Neither surface uses embeddings or a vector database, with only 6 stands the entire fleet's current state trivially fits in a single prompt, so retrieval search solves a problem that does not exist here. Both fail gracefully to a deterministic, still data grounded response if the AI provider is unreachable, the console never breaks because a network call did.



The fleet copilot answering a free text question about which stand is running hottest right now. That number came from an actual tool call into the live data at the moment the question was asked, not a stored or scripted reply.

How to run it

The full setup, both Python and frontend, is in the code deliverable's README. Two commands generate and score the dataset, two more start the backend and frontend, and the console is live at localhost.